CAHIER DES CHARGES DÉTAILLÉ: Application Microservices pour la SFMC Bénin

1. Présentation générale du projet

1.1 Contexte organisationnel

La SFMC Bénin est une entreprise industrielle spécialisée dans la production et la distribution de matériaux de construction (ciment, fer, briques, granulats). Les processus actuels sont partiellement manuels ou reposent sur des systèmes hétérogènes, entraînant :

- une faible traçabilité
- des erreurs de gestion de stock
- un manque de visibilité globale

1.2 Vision du système cible

Mettre en place une **plateforme numérique modulaire** reposant sur une architecture microservices permettant :

- une gestion distribuée des processus métier
- une interopérabilité entre services
- une forte scalabilité et résilience

2. Analyse des besoins métiers

2.1 Processus métier principaux

- **Chaîne de production**
  - Approvisionnement en matières premières
  - Transformation (fabrication)
  - Contrôle qualité
  - Mise en stock des produits finis
- **Chaîne commerciale**
  - Gestion des clients
  - Prise de commande
  - Facturation
  - Livraison
- **Chaîne logistique**
  - Gestion des stocks
  - Gestion des entrepôts

- Suivi des mouvements

2.2 Cas d'utilisation (exemples détaillés)

✅ **Cas 1 : Création d'une commande**

1. Le client passe une commande
2. Le système vérifie le stock
3. Si disponible → validation
4. Sinon → déclenchement production

✅ **Cas 2 : Mise à jour du stock**

- Entrée : production terminée
- Sortie : vente ou perte
- Synchronisation automatique avec Inventory Service

🔹 3. Architecture microservices (niveau avancé)

3.1 Décomposition par domaine (DDD - Domain Driven Design)

Domaine	Microservice
Identité	Auth Service
Utilisateurs	User Service
Catalogue	Product Service
Stock	Inventory Service
Commandes	Order Service
Production	Production Service
Finance	Billing/Payment Service
Communication	Notification Service
Analyse	Reporting Service

3.2 Description détaillée des microservices

🔐 **Auth Service**

- Authentification (JWT, OAuth2)
- Gestion des tokens
- Sécurité inter-services

👤 **User Service**

- CRUD utilisateurs
- Gestion des rôles (Admin, opérateur, client)

📦 **Inventory Service**

- Gestion multi-entrepôts
- Historique des mouvements
- Algorithme de seuil critique

🏭 **Production Service**

- Planification intelligente
- Suivi en temps réel
- Gestion des ressources machines

🛒 **Order Service**

- Gestion du cycle de vie des commandes
- États : pending, validated, shipped

3.3 Communication inter-services

🔹 **Synchrone (REST API)**

- Utilisé pour requêtes immédiates
- Exemple : Order → Inventory

🔹 **Asynchrone (Event-driven)**

- Utilisation de broker (Kafka/RabbitMQ)
- Exemple :
  - "OrderCreated"
  - "StockUpdated"

👉 Avantage : découplage fort et meilleure scalabilité

3.4 API Gateway

- Point d'entrée unique
- Routage vers microservices
- Gestion sécurité (authentification centralisée)

3.5 Service Discovery

- Enregistrement dynamique des services
- Exemple : Eureka / Consul

🔹 4. Modélisation des données

4.1 Principe

👉 **Base de données par microservice (Database per Service)**

4.2 Exemple de schéma (Inventory)

Stock

- id

- product_id

- quantity

- warehouse_id

StockMovement

- id

- type (IN/OUT)

- quantity

- date

4.3 Cohérence des données

- Cohérence éventuelle (eventual consistency)
- Saga pattern pour transactions distribuées

🔹 5. Sécurité (niveau expert)

5.1 Authentification

- JWT (stateless)
- OAuth2 pour intégration externe

5.2 Autorisation

- RBAC (Role-Based Access Control)

5.3 Sécurité réseau

- HTTPS obligatoire
- API Gateway sécurisé

5.4 Sécurité applicative

- Protection XSS, CSRF, injection SQL

🔹 6. Exigences non fonctionnelles (détaillées)

Critère	Exigence
Performance	< 2s
Disponibilité	99.9%
Scalabilité	horizontale
Maintenabilité	microservices indépendants
Sécurité	conforme standards OWASP

7. Planning détaillé

Phase	Activités	Durée
Analyse	Recueil besoins	2 jours
Conception	UML + architecture	3 jours
Développement	Microservices	4 jours
Tests	Unitaires + intégration	1 jours
Déploiement	Production	1 jours

8. Risques techniques (approfondi)

- Complexité de la communication inter-services
- Gestion des transactions distribuées
- Latence réseau
- Synchronisation des données

9. Facteurs clés de réussite

- Bonne définition des domaines métier (DDD)
- Tests automatisés
- Monitoring efficace
- Sécurité robuste